ConvNeXt and ViT for Hierarchical Open-Set Image Classification

Yitong Bai
yb2636@columbia.edu

Hsuan-Ting Lin
hl3930@columbia.edu

Peixin Shi
ps3255@columbia.edu

Abstract

We study hierarchical open-set image classification on a low-resolution dataset with three super-classes (bird, dog, reptile) and multiple sub-classes. We propose a two-stage transfer learning framework that first predicts the super-class and then routes each image to a super-class-specific sub-class head, using ImageNet-pretrained ConvNeXt and ViT backbones for feature extraction. To handle unknown categories at test time, we integrate open-set rejection via OpenMax and compare it against a simple Softmax confidence threshold baseline. On a proxy split that holds out both a novel super-class and novel sub-classes, our approach substantially improves unseen recognition over the CLIP baseline; ConvNeXt is more stable and achieves stronger overall accuracy than ViT. Notably, Softmax thresholding matches or surpasses OpenMax in this hierarchical setting, making it a competitive lightweight OSR strategy.

1 Introduction

In recent years, deep learning has achieved remarkable success in closed-set image classification tasks. However, in real-world application scenarios, intelligent systems often face more complex challenges: not only do they need to recognize objects with taxonomic hierarchy structure[10] (Yan et al., 2015), but also must have the ability to handle unseen categories. For example, an ecological monitoring system not only needs to distinguish between "birds" and "reptiles", but also needs to issue an alarm when encountering new species that are not in the training set, rather than mistakenly classifying them as known species[1] (Bendale & Boult, 2016).

This study focuses on the hierarchical open-set classification task for low-resolution images. This task faces two core challenges: first, hierarchical dependence, where the model needs to capture the features of both coarse-grained super-classes and fine-grained sub-classes, while flat classifiers often ignore this inherent connection; second, open-world constraints, where the unseen categories in the test set cause traditional closed-set SoftMax classifiers to exhibit overconfidence and be difficult to effectively reject.

To address these issues, we propose a two-stage hierarchical classification framework combined with transfer learning. Using the pre-trained backbone networks (ConvNeXt and ViT) from ImageNet to extract general visual features[8] (Liu et al., 2022), we adopt a "coarse-to-fine" reasoning strategy: in the first stage, we predict the super-classes, and in the second stage, we conditionally predict the sub-classes based on the results of the super-classes. To solve the open-set recognition (OSR) problem, we integrate the OpenMax mechanism in the classification head, which models the distribution of activation vectors to calibrate the predicted probabilities, thereby achieving effective detection of new super-classes and sub-classes[1] (Bendale & Boult, 2016).

In our empirical study, we additionally evaluate a simple Softmax-based confidence thresholding strategy for open-set detection. Surprisingly, we observe that Softmax confidence performs on par

Foundation Models for Science Workshop,38th Conference on Neural Information Processing Systems (NeurIPS 2024).

with, and in some cases better than, OpenMax in this hierarchical setting, suggesting that simpler confidence-based rejection can be a strong baseline.

The main contributions of this paper are summarized as follows:

- Proposed a two-stage hierarchical framework: By leveraging hierarchical dependencies, a transfer learning architecture was designed, effectively solving the classification problem of low-resolution images.
- Baseline evaluation of backbone networks: Systematically compared the performance and computational costs of ConvNeXt [8] and ViT in hierarchical tasks.
- Integration of open-set recognition: Introduced OpenMax to implement a dual-level rejection mechanism, significantly enhancing the model's robustness to unknown classes.
- Verification of distribution shift: By analyzing the distribution differences between known and unknown classes, it was confirmed that this method is effective in open-world scenarios.

2 Related Work

Our project targets two core challenges: hierarchical image classification and open-set recognition (OSR). We review prior work on (1) multi-stage hierarchical prediction and (2) rejecting novel categories at test time.

Hierarchical image classification. A common strategy for hierarchical labeling is a multi-stage or cascaded model that decomposes prediction into coarse-to-fine decisions. For example, HMIC uses a parent model to identify a coarse disease category and a child model to refine severity, allowing each stage to specialize on features relevant to its level [7]. Similar two-step designs have also been explored in medical imaging tasks where a first stage determines a broader condition and a second stage refines the subtype [5]. Beyond medical settings, hierarchical prediction has been applied to large-scale classification systems, including e-commerce category prediction [3]. In addition, transfer learning can be particularly helpful when fine-grained classes have limited data; a recent study on Amazon parrot classification reports that hierarchical transfer learning outperforms a non-hierarchical baseline [6]. These findings motivate our two-stage design: super-class prediction followed by conditional sub-class prediction.

Backbone networks. We use ImageNet-pretrained backbones to extract general visual representations and compare a modern convolutional architecture (ConvNeXt) and a transformer-based model (ViT). ConvNeXt revisits ConvNet design choices and demonstrates strong performance as a pure convolutional backbone [9]. ViT represents an image as a sequence of patch tokens and applies a Transformer encoder, achieving competitive transfer learning performance [2]. These two backbones provide complementary inductive biases for our hierarchical heads.

Open-set recognition. Our evaluation includes novel super-classes and novel sub-classes at test time, requiring the model to reject unknown inputs. A survey on OSR highlights that standard closed-set classifiers with SoftMax are not designed for unknown categories and can be over-confident on unseen data [4]. To address this, we adopt OpenMax, which calibrates class probabilities by modeling activation-vector distances to class mean activation vectors and fitting a Weibull distribution to the tail, reallocating probability mass to an unknown class [1]. This fits our setting because we need an explicit "novel" decision while keeping the main classifier structure unchanged.

3 Method

This section is a description of our two-stage hierarchical classification framework method, including the backbone network used for feature extraction, the super-class prediction module, the conditional sub-class prediction module, and the Open Set Recognition(OSR) via OpenMax. The whole training and inference pipeline is shown in Figure 1.

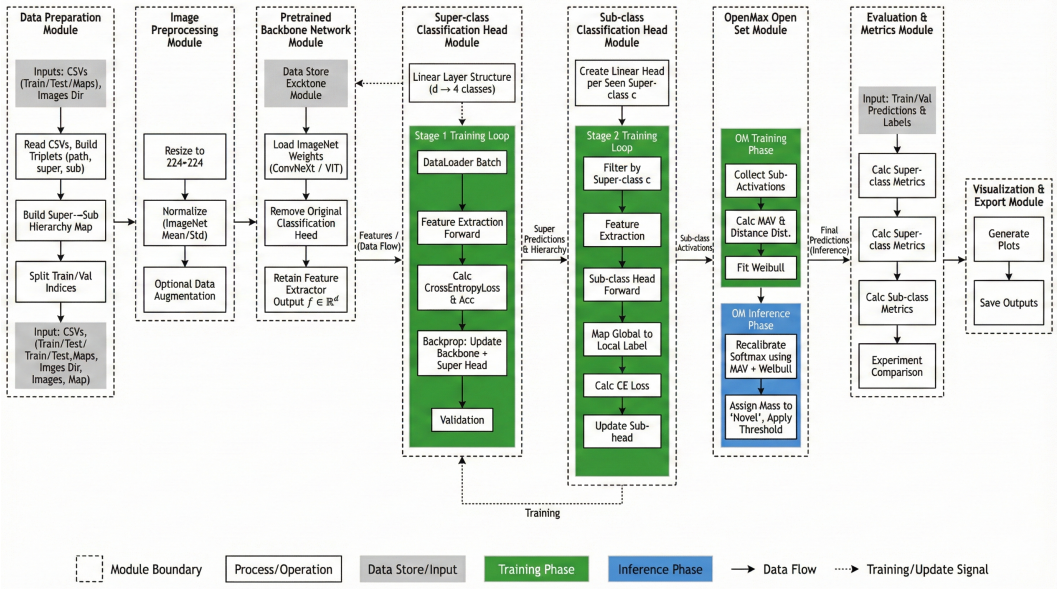

Figure 1: Overall training and inference pipeline.

3.1 Problem Formulation

The hierarchical classification structure is designed for this task, which has two levels of labels, a coarse super-class and a sub-class with fine-grained. The dataset contains three superclasses: bird, dog, and reptile, each consisting of multiple subclasses. The model must output both labels for every input image.

Each input sample consists of an image and two labels: a super-class label and a fine-grained sub-class label. Let the input image be

$$x \in \mathbb{R}^{3\times 64\times 64}.$$

The super-class label space is

$$y^{(s)} \in \{1, 2, \ldots, C_s\} \cup \{\text{novel}\},$$

and the sub-class label space is

$$y^{(c)} \in \{1, 2, \ldots, C_c\} \cup \{\text{novel}\}.$$

During testing, unseen novel super-classes and novel sub-classes appear; hence the model performs hierarchical prediction in an open-set scenario.

A unified training objective can be written as

$$\mathcal{L} = \mathcal{L}_{\text{super}} + \sum_{c=1}^{C_s} \mathbb{1}(y^{(s)} = c)\, \mathcal{L}_{\text{sub}}^{(c)},$$

where $\mathcal{L}_{\text{super}}$ is the super-class cross-entropy loss and $\mathcal{L}_{\text{sub}}^{(c)}$ is the sub-class loss for super-class c.

Evaluation follows the official metrics of the competition, including super-class accuracy, sub-class accuracy, super- and sub-class categorical cross-entropy, and accuracy on seen versus unseen (novel) categories at both levels.

3.2 Backbone Networks

We adopt two different ImageNet-pretrained models, ConvNeXt and Vision Transformer (ViT), as backbone feature extractors, aiming to compare their performance. An input X is mapped to a shared

feature representation:

$$h(x) = f_\theta(x) \in \mathbb{R}^d,$$

where f_θ denotes the pretrained backbone with output dimensionality d.

3.2.1 ConvNeXt Backbone

ConvNeXt is a modern convolutional architecture that has a similar design to ResNet. During the forward pass, after processing through multiple convolutional layers, images will be applied to an average pooling layer:

$$h(x) = \text{GAP}(\text{ConvNeXtBlocks}(x)).$$

We remove the original classification head from the pretrained model so that only the feature extractor is retained.

3.2.2 Vision Transformer (ViT) Backbone

ViT turns an image into tokens, then processes them by Transformer encoder The output corresponding to the CLStoken serves as the global image representation:

$$h(x) = \text{CLS}(\text{TransformerEncoder}(x)).$$

Same as ConvNeXt, we replace the original classification head with an identity mapping to expose the feature.

3.3 Two-Stage Hierarchical Classifier

To finish the requirement of task requirement, we designed a Two-Stage Hierarchical Classifier. 1) predicting the super-class; 2) predicting the sub-class conditioned on the Stage-1 super-class output. The Flow Chart of that classifier is shown in the Figure 2.

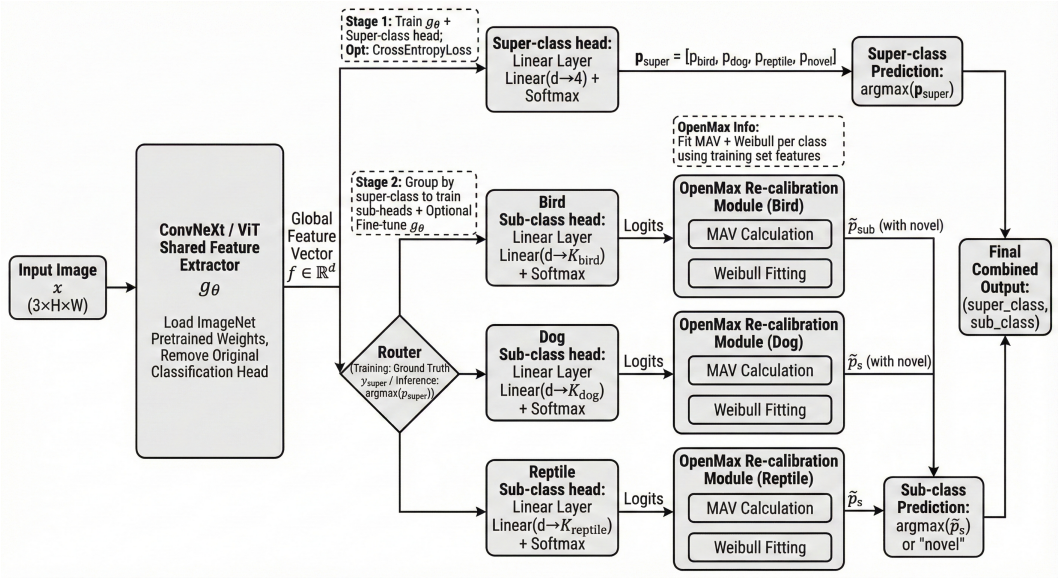

Figure 2: Two-stage hierarchical classifier.

3.3.1 Stage 1: Super-class Prediction

This stage predicts the super class label among bird, dog, reptile, novel, where "novel" is reserved for testing. The backbone feature h(x) extracted from Section 3.2 will be the shared input to the super-class head.

$$z^{(s)} = W^{(s)} h(x), \qquad p^{(s)} = \text{softmax}(z^{(s)}).$$

The super-class loss is

$$\mathcal{L}_{\text{super}} = -\log p^{(s)}_{y^{(s)}}.$$

Backbone parameters θ and the super-head parameters ϕ are updated jointly. The model is trained in the backbone, and the super-class head in this section, and the best model will be saved for the next stage.

3.3.2 Stage 2: Conditional Sub-class Prediction

Within the code, we maintain a `global super-sub-id` list encompassing all subclasses under a superclass for subclass "routing". For each superclass, we construct a subclass header, with each header trained independently to reduce confusion and address the problem statement.

$$z^{(c)}_k = W^{(c)}_k h(x), \qquad p^{(c)}_k = \text{softmax}(z^{(c)}_k),$$

Here, $k \in \{\text{bird}, \text{dog}, \text{reptile}\}$ indexes the super-class-specific sub-head. Each head $W^{(c)}_k$ predicts only the sub-classes belonging to the super-class k.

$$\mathcal{L}_{\text{sub},k} = -\mathbb{E} \log p_{k,\, y^{(c)}}.$$

Sub-class loss is computed only for samples whose ground-truth super-class corresponds to the head.

3.3.3 Conditional Routing

a.training routing: the model uses the truth super-class $y^{(s)}$ to select the appropriate head $W^{(c)}_{y^{(s)}}$, ensuring that each sub-head is updated only with relevant samples.

b.Inference routing: Stage-1 outputs a predicted super-class k, which determines which sub-head $W^{(c)}_k$ is applied to produce the final subclass prediction.

3.3.4 Algorithm 1: Two-stage Hierarchical Training Procedure

The complete training is summarized in Algorithm 1, Algorithm 2, and Algorithm 3:

Algorithm 1 Stage 1: Super-Class Training

Require: Training set $\mathcal{D}$, backbone g_θ, super-head h_ϕ, epochs T_1, lr η_{super}, batch size B
Ensure: Updated (g_θ, h_ϕ)

```
Initialize h_φ
Split dataset into train/val
Set Adam optimizer on (θ, φ)
for t = 1 to T_1 do
    for mini-batch (x_b, y_b^(s)) do
        f_b = g_θ(x_b)
        z_b^(s) = h_φ(f_b)
        L = CE(z_b^(s), y_b^(s))
        Update (θ, φ) by Adam
    end for
    Evaluate on validation set and save best checkpoint
end for
```

Algorithm 2 Stage 2: Sub-Class Training for Each Super-Class

Require: Backbone g_θ, sub-heads $\{k_{\psi_c}\}$, training set, epochs T_2
Ensure: Trained $\{k_{\psi_c}\}$

for each super-class c **do**
  Build sub-dataset $\mathcal{D}_c$
  Set optimizer over ψ_c (and optionally θ)
  for $t = 1$ to T_2 **do**
    for mini-batch $(x_b, y_b^{(c)})$ **do**
      $f_b = g_\theta(x_b)$
      $z_{b,c} = k_{\psi_c}(f_b)$
      $L_c = \mathrm{CE}(z_{b,c}, y_{b,c}^{\text{local}})$
      Update parameters
    end for
  end for
end for

Algorithm 3: Stage 3 OpenMax Fitting for Open-Set Recognition

Require: Trained sub-heads, training set, tail-size
Ensure: Weibull parameters $(\mu_k, \lambda_k, \kappa_k)$ for each seen class

for each seen class k **do**
  Collect activations $\{a_j\}$
  Compute MAV:

$$\mu_k = \frac{1}{|\{a_j\}|} \sum_j a_j$$

  Compute distances $d_j = \|a_j - \mu_k\|_2$
  Select largest distances and fit Weibull:

$$(\lambda_k, \kappa_k) = \mathrm{WeibullFit}(d_j)$$

end for

3.4 Open Set Recognition with OpenMax

In practice, we implement and compare two open-set recognition strategies: OpenMax-based recalibration and a Softmax confidence thresholding baseline. Experimental results show that the Softmax-based approach yields competitive and sometimes superior performance compared to OpenMax, motivating its use as a strong and lightweight alternative in our framework.

3.4.1 OpenMax Recap

To handle "novel" classes during inference, we adopt the OpenMax framework, which models the "distance from known classes" using activation vectors and recalibrates class probabilities accordingly. For each seen class k, let $a_i \in \mathbb{R}^d$ denote the activation vector of sample i. The Mean Activation Vector (MAV) is computed as:

$$\mu_k = \frac{1}{N_k} \sum_{i:\, y_i = k} a_i.$$

Distances between activations and the class MAV are defined as:

$$d_i = \|a_i - \mu_k\|_2.$$

OpenMax fits a Weibull CDF to the tail of this distance distribution:

$$F_k(d) = 1 - \exp\left[-\left(\frac{d}{\lambda_k}\right)^{\kappa_k}\right],$$

where λ_k and κ_k denote the scale and shape parameters. The resulting CDF quantifies how atypical an activation is with respect to class k, enabling probability mass to be reallocated to a novel category.

3.4.2 Integration into the Hierarchical Framework

At the super-class level, before applying OpenMax, we perform a lightweight unseen-class check using the raw softmax output of the super-class head. Let

$$p^{(s)} = \mathrm{softmax}(z^{(s)})$$

represent the predicted super-class probability, and let

$$p_{\max} = \max_j p_j^{(s)}$$

denote its maximum confidence.

We follow a simple thresholding rule:

> If $p_{\max} < \tau_{\text{super}}$, the sample is predicted as a novel super-class. Otherwise, we treat the predicted class as seen and proceed to Stage 2.

In Stage 2, once a specific sub-class head is selected according to the predicted super-class, OpenMax is applied to that head to detect a novel sub-class, ensuring hierarchical consistency.

3.4.3 Inference Recalibration

For the sub-class head associated with super-class s, let the original softmax probability for a seen class y be $p(y)$. OpenMax attenuates this probability using the Weibull CDF:

$$\tilde{p}(y) = p(y)\left(1 - F_k(d)\right), \qquad d = \|a - \mu_k\|_2,$$

where d is the distance between the sample's activation vector and the MAV of class k.

The remaining probability mass is assigned to the novel sub-class:

$$p(\text{novel}) = 1 - \sum_{y \in \text{seen}} \tilde{p}(y).$$

A sample is classified as a novel sub-class if

$$p(\text{novel}) \geq \tau_{\text{sub}},$$

where τ_{sub} is a threshold selected via validation.

Thus, Stage 1 filters novel super-classes via softmax confidence, whereas Stage 2 detects novel sub-classes via OpenMax recalibration, forming a unified two-level open-set prediction pipeline.

4 Experiments

4.1 Dataset and splits

4.1.1 Competition dataset

We use the competition dataset containing 6,288 images, each with resolution 64×64. There are three super-classes: Bird, Dog, and Reptile. Each super-class contains 29 sub-classes (e.g., hawk under Bird, golden retriever under Dog), resulting in 87 sub-classes in total. The class distribution is relatively balanced, with 1,850 Bird images, 2,084 Dog images, and 2,354 Reptile images.

4.1.2 Local validation design

To simulate the Open-Set Recognition (OSR) requirements of the competition, we designate specific categories as *novel* and exclude them entirely from the training phase. Novelty is defined at two hierarchical levels. At the super-class level, the entire Dog category (super-class ID 1) is treated as the novel super-class. At the sub-class level, we hold out four sub-classes as novel: Great Grey Owl (sub-class ID 4) and Magpie (sub-class ID 84) from Birds, and African Chameleon (sub-class ID 1) and Triceratops (sub-class ID 52) from Reptiles.

Table 1: Official baseline performance (CLIP/B-32). "Seen" refers to known classes, and "Unseen" refers to novel classes in our proxy split.

Model	Cross-entropy (↓)		Super-class Acc (%) (↑)			Sub-class Acc (%) (↑)		
	Super	Sub	Overall	Seen	Unseen	Overall	Seen	Unseen
CLIP/B-32	1.36	4.44	80.42	99.14	32.91	13.10	60.75	0.24

These definitions enable a proxy split for local evaluation. The training set consists of 80% *known* data, including Bird and Reptile images except for the four held-out sub-classes, so the model learns only from known categories. The validation set contains the remaining 20% known data, together with a balanced subset of 300 Dog images and 200 held-out sub-class images. This design supports threshold tuning, because it tests whether the model can recognize known examples while rejecting novel inputs.

4.2 Baselines

We use the official competition baseline, CLIP/B-32, as a reference point for evaluation. As shown in Table 1, the baseline achieves cross-entropy losses of 1.36 for super-classes and 4.44 for sub-classes. The baseline performs strongly on known data (99.14% super-class accuracy on seen classes), but performs poorly on novel data (32.91% accuracy on unseen super-classes and 0.24% accuracy on unseen sub-classes). This gap highlights the difficulty of OSR and motivates methods that explicitly handle novelty.

4.3 Implementation details

Hyperparameters and training strategy. We train using the Adam optimizer with a batch size of 16. To match the hierarchical label structure, we adopt a two-stage training strategy. In Stage 1 (super-class training), we train for 5 epochs with learning rate 1×10^{-4} to stabilize the backbone and learn coarse features. In Stage 2 (sub-class fine-tuning), we train for 10 epochs with learning rate 1×10^{-5} to refine features while reducing catastrophic forgetting. During Stage 2, we enable backbone updates (`finetune-stage2`) so the backbone can adapt to fine-grained classes. We use a validation split of 0.2 and save the best checkpoint based on validation accuracy.

Hardware environment. All experiments are conducted on Google Colab using a single NVIDIA A100 Tensor Core GPU.

Code structure and reproducibility. Our implementation is based on PyTorch. The training pipeline is organized into modular scripts (e.g., `train_vit.py` and `train_convnext.py`) and supports different backbones such as `vit_b_16` and `convnext_tiny`. We implement a custom `HierarchicalClassifier` that contains two classification heads, one for super-classes and one for sub-classes. Training uses a joint loss to encourage hierarchical consistency.

5 Results

5.1 Overall Performance on Proxy Validation Set

Table 2: Overall Performance on Proxy Validation Set

Model	Cross-entrophy		Superclass accuracy(%)			Subclass accuracy(%)		
	Super class	Sub class	overall	Seen	Unseen (novel)	overall	seen	unseen
CLIP/B-32	1.36	4.44	80.42	99.14	32.91	13.10	60.75	0.24
1. ConvNeXt-Tiny (Super: Softmax, Sub: OpenMax)	0.0033	0.4905	93.1	95.11	86.87	68.99	75.16	59.6
2. ConvNeXt-Tiny (Super: Softmax, Sub: Softmax)	0.0033	0.4905	93.1	95.11	86.87	85.25	81.08	91.60
3. ViT-B/16 (Super: Softmax, Sub: Softmax)	0.0431	0.6374	87.55	87.1	88.0	77.54	76.48	78.6

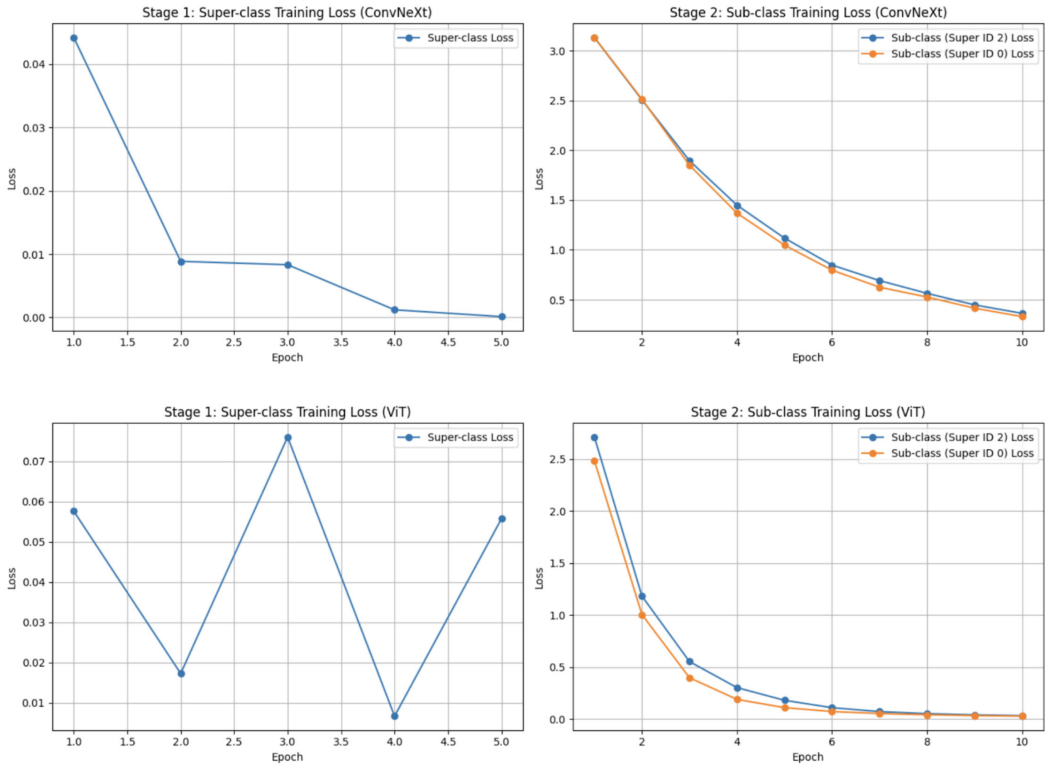

Figure 3: Training loss curves for ConvNeXt-Tiny and ViT-B/16 in Stage 1 (super-class) and Stage 2 (sub-class).

Performance Analysis First of all, all three custom models dominate the CLIP/B-32 baseline on Open-Set Recognition tasks: while the baseline is nearly unable to predict unseen sub-class labels (0.24% accuracy), our models show a robust rejection capability that lead to over 90% accuracy in predicting with unseen sub-clases (open classes). With respect to the backbone architecture, ConvNeXt-Tiny (Model 2) outperforms ViT-B/16 (Model 3) across several orders of magnitude in final Superclass cross-entropy loss (0.0033 vs. 0.0431), and achieves higher Overall Subclass accuracy (85.25% vs. 77.54%), demonstrating its advantage both for recognizing known samples and rejecting novel ones. Lastly, in terms of inference technique, the Softmax Thresholding (Model 2) was significantly better than OpenMax (Model 1) for this dataset. While both had equivalent "Seen" performance Softmax attained a 91.60% accuracy on the unseen sub-classes, versus OpenMax's 59.60%, indicating that just using probability thresholds was more appropriate in this particular proxy split situation. To conclude, Model 2 (ConvNeXt-Tiny + Softmax) is the best compromise between known-class performance and novel-category defense and the subsequent analysis will focus on the top two performers, Model 2 and Model 3.

5.2 Backbone comparison: ConvNeXt vs ViT

Training Stability & Convergence As shown in Figure 3,ConvNeXt-Tiny (CNN) presents a better stability, as the loss curves of both super-class (Stage 1) and sub-class (Stage 2), smoothly decreases monotonically; it suggests that CNN's inductive biases such as translation invariance, locality etc., are suitable to small dataset of size 64x64 and learn discriminatively useful features without oscillating. In contrast, ViT-B/16 (Transformer) presents severe instability: it suffers from several order-of-magnitude fluctuations in the loss curve in Stage 1, implying a challenge to aggregate reliable attention maps over limited data, and rapidly snaps to near-zero loss in Stage 2 — a typical symptom of overfitting where the model learns to recite training samples instead of capturing generalizable features that generalize poorly on new test data.

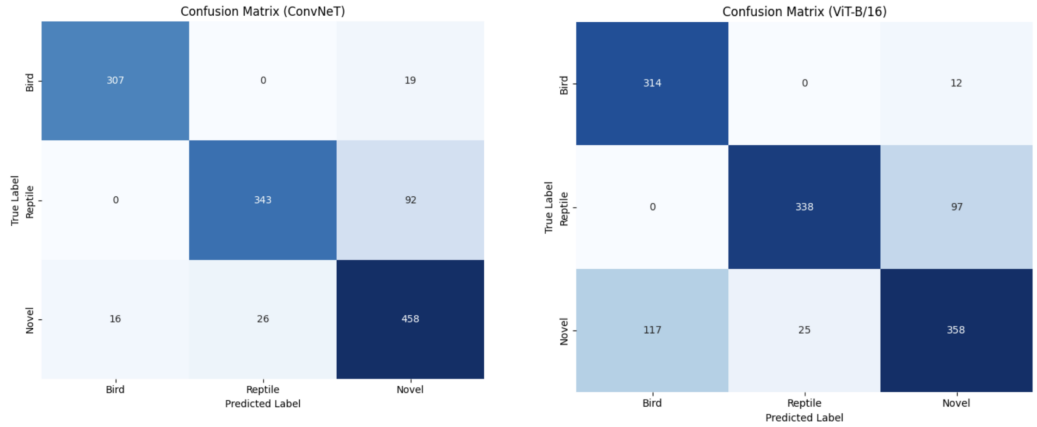

Figure 4: Super-class confusion matrices on the proxy validation set for ConvNeXt-Tiny and ViT-B/16.

5.3 Per-class analysis and error breakdown

5.3.1 Super Class Analysis

As shown in Figure 4, ConvNeXt-Tiny excels at Open-Set with strong novelty rejection, almost perfectly classifying most of "Novel" (Dog) images as "Novel", and keeping a clean, strict boundary between the known Birds and Reptiles. In contrast, ViT-B/16 suffers from catastrophic safety critical failures arising from high confidence, i.e., rather than rejecting novel examples it consistently incorrectly classifies them as non-novel classes ("False Knowns"), which further suggests that it memorized the closed-set distribution instead of learning a robust feature space. In summary, ConvNeXt is the best backbone for this task and provides a safe and reliable flagging system of unknown inputs.

5.3.2 Sub-class analysis

Both ConvNeXt and ViT models repeatedly achieve strong performance on Birds ($> 90\%$ accuracy) and have some difficulty on Reptiles ($\sim$ 75–80%), as shown in Figure 5, a difference strongly influenced by two primary facts. Firstly, in relation to Shape vs. Texture: birds have very distinctive silhouettes (e.g., long necks and wings) that are still recognizable at low res, whereas many reptiles can have a generic "lizard-like" body plan and rely on fine texture details (e.g., scales) that are lost at this resolution. The second is that with respect to Camouflage vs. Contrast, reptiles are biologically selected to match more complex backgrounds, which makes for failures in segmentation and negative detection (particularly so is birds are often captured against high-contrast backgrounds such as sky or water). These findings suggest that both architectures rely largely on the distinct shape cues for classification — which happen to lean in the direction of Birds — over the small-scale, highfrequency texture details that are necessary to differentiate Reptiles.

5.4 overall performance on test set (leader board)

Table 3: overall performance on test set (leader board)

Model	Superclass accuracy(%)			Subclass accuracy(%)		
	overall	Seen	Unseen (novel)	overall	seen	unseen
1.CLIP/B-32 (baseline)	80.42	99.14	32.91	13.10	60.75	0.24
2. ConvNeXt-Tiny (Super: Softmax, Sub: Softmax)	81.46	85.89	70.22	78.25	63.53	82.22
3. ViT-B/16 (Super: Softmax, Sub: Softmax)	81.31	88.28	63.61	64.93	62.68	65.53

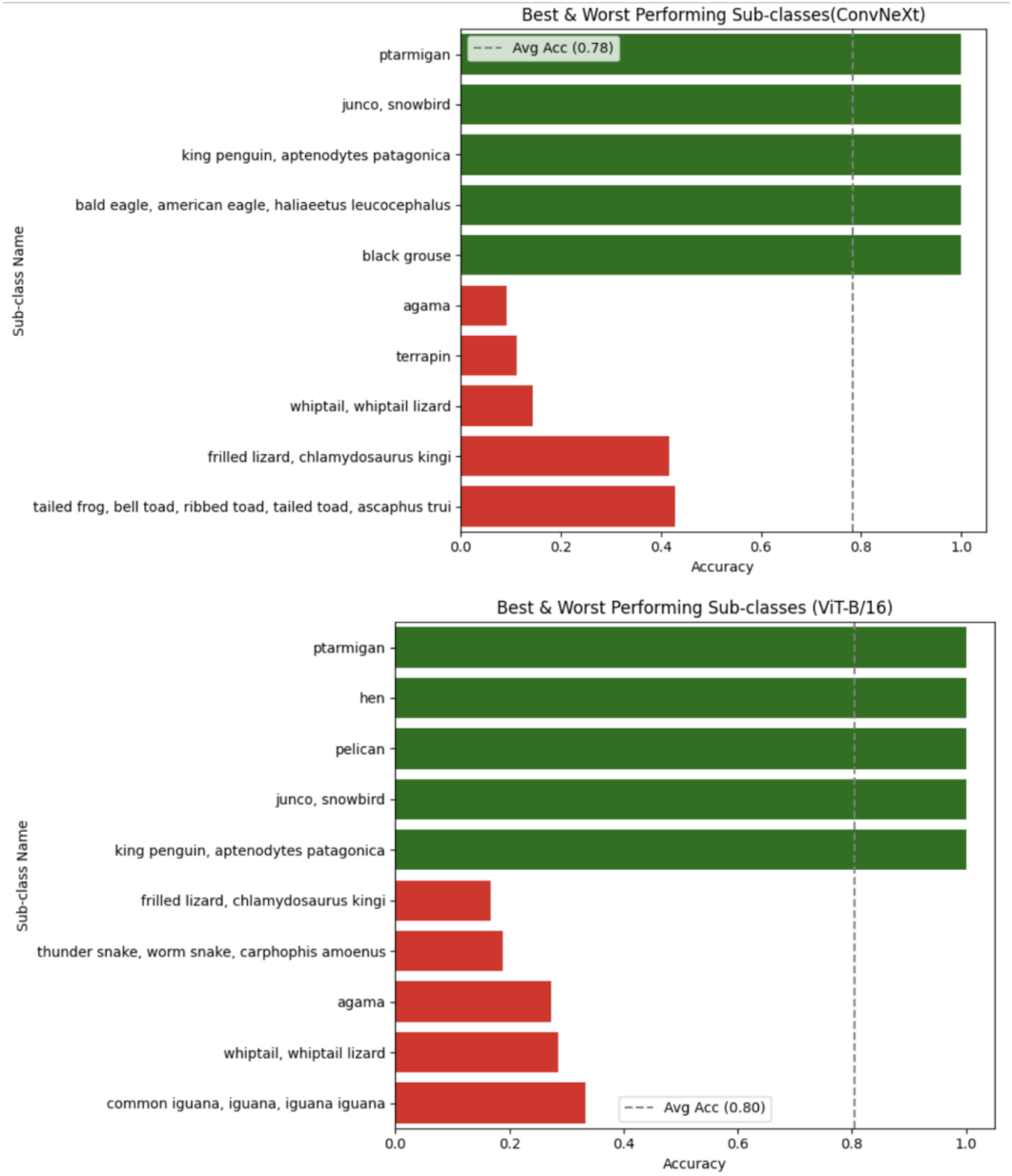

Figure 5: Best and worst performing sub-classes for ConvNeXt-Tiny and ViT-B/16 on the proxy validation set. The dashed line indicates the average sub-class accuracy.

ConvNeXt-Tiny also performs well in OSR with top Unseen Sub-class accuracy (82.22%) and significantly outperforms both ViT (65.53%) and CLIP baseline (0.24%), which suggests robust, generalizable features are learned as opposed to merely memorizing the training set by CNN-based architecture. On the other hand, ViT-B/16 reaches a competitive "Seen" Super-class accuracy (88.28%), but generalizes poorly as it plummets down to 65.53% on unseen sub-classes, suggesting an overconfident behaviour of mistakenly classifying something new as seen classes. These findings coincide with the training dynamics studied in Section 5.2, affirming that the ViT model faced overfitting and optimization instability, while ConvNeXt traded off precision on known categories against novelty rejection robustness.

6 Discussion and Limitations

This imposed two-stage hierarchical architecture was particular useful for "Seen" categories, as it allowed the model to first specialize on well-separated global shapes (Bird vs. Reptile), before incorporating detailed-texture information. But, it also introduces one of errors propagating: Error Propagation. When a global feature is mis-classified (e.g., classifying camouflaged Reptile to Bird), Stage 2 classifier has to predict the semantic sub-space owned by other space and it can never be recovered. To address this "hard decision" problem, future work could explore Joint Training with multi-task loss or "Soft Routing" for the two heads which are able to back-propagate gradients through both heads jointly to learn jointly robust shared features on different level of abstraction.

6.1 Effectiveness and Limits of Softmax Thresholding

Softmax Thresholding performed remarkably consistent at the levels of both Super and Sub-classes, meaning that for this particular domain a well-trained CNN raw confidence score is a good predictor of novelty. However, this approach is still very sensitive to the hyperparameter; a high threshold results in improved rejection rates but a significantly lower recall for "hard" or unknown classes e.g., camouflaged reptiles which naturally exhibit lower confidence scores. Compared to OpenMax, which assumes the features from known and unknown classes follow one distribution (Weibull), that may not be the case for small low resolution datasets, Softmax merely relies on the entropy of the prediction is less theoretic because of this, but more robust in practice to whatever reasons our particular split of data deviates from its assumptions.

6.2 Why OpenMax Performed Poorly?

The large drop in performance for OpenMax (59.60% unseen accuracy) with respect to Softmax (91.60%) implies that Feature Collapse can be a critical factor limiting the reliability of the features' scores. Deep networks trained with standard Cross-Entropy loss will compact autantically similar samples from the same class into ultra-tight clusters or single direction rays to maximize separability, effectively killing off the intra-class variance repoired by OpenMax's Weibull tail-fitting. The computed distance distributions become degenerate when merging features to a point, and in turn OpenMax fails to build valid probability models for these outliers; future work could target this problem by featuring specialized objectives like Contrastive Loss or Center Loss that could enforce the required geometric spread into the embedding space.

7 Conclusion and Future Work

We show that a two-stage hierarchical transfer learning approach significantly outperforms zero-shot baselines for Open-Set Recognition, with unseen sub-class accuracy improved from 0.24% (CLIP) to more than 82% (ConvNeXt-Tiny). Our experiments reveal that the ConvNeXt-Tiny is the better backbone, and the inductive bias of it played an important role for stability when using this dataset with small size and low resolution, compared to optimization instability and overfitting in ViT-B/16. In addition, non-convoluted Softmax Thresholding was shown to perform better than the more complicated OpenMax method, which most likely occurred because of feature collapse within the embedding space which made it difficult for Weibull distributions to fit effectively.

In the future, we need to convert the sequential training to End-to-End Joint Hierarchical Training for reducing error propagation between super-class and sub-class. Furthermore it is also possible to improve the robustness of the system by switching to Energy-based or Generative Models for a more refined outlier rejection and discarding heuristic thresholds. Finally, effectively constructing harder classes (e.g., Reptiles) to rectify the performance drop in the imbalance condition, such a challenge can be addressed by introducing Class-Balanced Loss or Meta-Learning for enhancing generalization on few-shot and imbalanced setting.

References

[1] Abhijit Bendale and Terrance E. Boult. Towards open set deep networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1563–1572, 2016. doi: 10.1109/CVPR.2016.173.

[2] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2010.11929.

[3] D. Gao, W. Yang, H. Zhou, Y. Wei, Y. Hu, and H. Wang. Deep hierarchical classification for category prediction in e-commerce system, 2020.

[4] Chuan Geng, Sheng-Jun Huang, and Songcan Chen. Recent advances in open set recognition: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(10):3614–3631, 2021. doi: 10.1109/TPAMI.2020.2981604.

[5] M. Hu, B. Wu, D. Lu, J. Xie, Y. Chen, Z. Yang, and W. Dai. Two-step hierarchical neural network for classification of dry age-related macular degeneration using optical coherence tomography images. *Frontiers in Medicine*, 10:1221453, 2023. doi: 10.3389/fmed.2023.1221453.

[6] J. I. Kim, J. W. Baek, and C. B. Kim. Hierarchical image classification using transfer learning to improve deep learning model performance for amazon parrots. *Scientific Reports*, 15:3790, 2025. doi: 10.1038/s41598-025-88103-3.

[7] Kamran Kowsari, Réza Sali, Laya Ehsan, William Adorno, Ali Ali, Steven Moore, Ben Amadi, Paul Kelly, Shahid Syed, and David Brown. Hmic: Hierarchical medical image classification, a deep learning approach. *Information*, 11(6):318, 2020. doi: 10.3390/info11060318.

[8] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11976–11986, 2022. doi: 10.1109/CVPR52688.2022.01167.

[9] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11966–11976, 2022. doi: 10.1109/CVPR52688.2022.01167.

[10] Zhicheng Yan, Hao Zhang, Raghuraman Piramuthu, Vignesh Jagadeesh, Dennis DeCoste, Wei Di, and Yizhou Yu. Hd-cnn: Hierarchical deep convolutional neural networks for large scale visual recognition. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2740–2749, 2015. doi: 10.1109/ICCV.2015.314.

8 Personal Contributions

Yitong Bai yb2636

- Implemented the basic model architecture, including backbone integration, shared utility modules, data builders, and training code for ConvNeXt and Vision Transformer, providing a unified foundation for subsequent experimental evaluation and model comparison.
- Wrote the Introduction and Methods sections of the paper.

Hsuan-Ting Lin hl3930

- Implemented experiments for ConvNeXt and ViT, including validation set design, hyperparameter tuning, performance analysis, final model training and predictions.
- Collaborated with Peixin Shi to organize and polish the Experiments, Results, Discussion, and Future Work sections.

Peixin Shi ps3255

- Debugged and experimented with `train_vit.py` to improve ViT training stability and performance; the results were not competitive, so this variant was not adopted in the final submission.

- Wrote the Related Work section, and collaborated with Hsuan-Ting Lin to organize and polish the Experiments, Results, Discussion, and Future Work sections (including tables/figures integration and overall write-up consistency).